Assignment No: 8

Title: Set up a local Kubernetes cluster with Minikube to deploy and scale your Dockerized AI application.

Steps:

Phase 1: Environment Setup

1. **Start the Cluster**: Initialize your local single-node Kubernetes cluster.

```
None
minikube start
```

2. **Verify Cluster Status**: Confirm the cluster is running correctly.

```
None
minikube status
```

Phase 2: Building the AI Application Image

You need to build a Docker image that the Kubernetes cluster can access.

1. **Connect to Minikube's Docker Engine**: Configure your terminal to use Minikube's internal Docker environment

```
None
eval $(minikube docker-env)
```

2. **Create Project Directory**: Organize your application files

```
None
mkdir exp8-k8s-ai && cd exp8-k8s-ai
```

3. **Create a simple AI app (app.py):**

```
None
        from flask import Flask app =
        Flask(__name__) @app.route('/')
        def home():
         return "AI Model is running on Kubernetes!" if __name__ ==
        "__main__": app.run(host='0.0.0.0', port=5000)
```

4. **Create a Dockerfile:**

```
None
        FROM python:3.9-slim RUN pip install
        flask COPY app.py . CMD ["python",
        "app.py"]
```

5. **Build the Image**: Create the Docker image for your AI application

```
None
docker build -t ai-model:k8s .
```

Phase 3: Deployment and Scaling

This phase involves defining the "Desired State" for your application and scaling it.

1. **Create Deployment Configuration**: Create a file named ai-deployment.yaml

with the following content to define your pods and replicas: YAML

None

```
apiVersion: apps/v1 kind:
Deployment metadata:
 name: ai-model-deployment
spec:
 replicas: 1 selector:
matchLabels: app: ai
model template:
metadata: labels:
 app: ai-model spec:
containers:
 - name: ai-model image: ai
model:k8s ports: -
containerPort: 5000
```

2. **Apply Deployment**: Use kubectl to create the application in the cluster

None

```
kubectl apply -f ai-deployment.yaml
```

3. **Expose the Service**: Create a service to make the AI application accessible outside the cluster

None

```
kubectl expose deployment ai-model-deployment --type=NodePort --port=5000
```

4. **Access the Application**: Open the AI app in your web browser.

None

```
minikube service ai-model-deployment
```

5. **Scale the Application**: Increase the number of replicas to handle more load.

```
None

kubectl scale deployment ai-model-deployment --replicas=3
```

6. **Verify Scaling**: Confirm that three pods are now running.

```
None

kubectl get pods
```

Phase 4: Cleanup

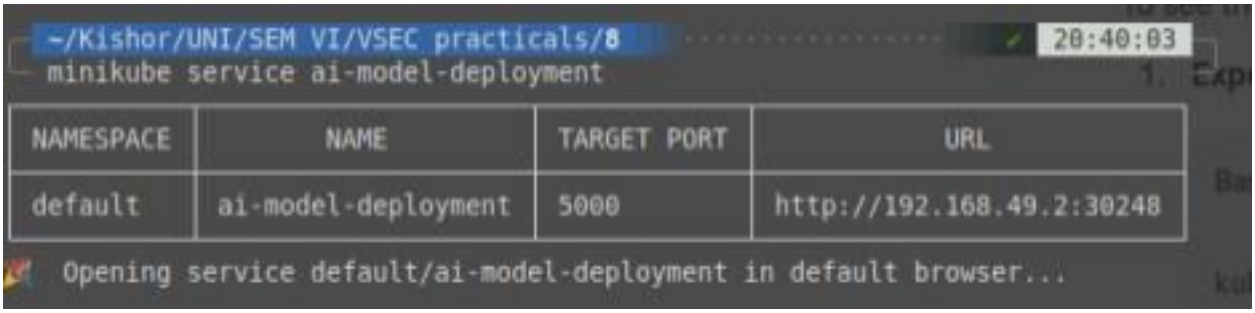
When you are finished with the experiment, stop the cluster to free up resources.

```
None

minikube stop
```

Output:

1. minikube service deployment



2. kubectl pods

```
~/Kishor/UNI/SEM VI/VSEC practicals/8                    INT x  20:40:35
 kubectl scale deployment ai-model-deployment --replicas=3
deployment.apps/ai-model-deployment scaled

~/Kishor/UNI/SEM VI/VSEC practicals/8                    ✓  20:40:37
 kubectl get pods
NAME                                  READY   STATUS    RESTARTS   AGE
ai-model-deployment-6fc47c4bbf-4ng5z  1/1     Running   0          47s
ai-model-deployment-6fc47c4bbf-jgblp  1/1     Running   0          5s
ai-model-deployment-6fc47c4bbf-mr2b5  1/1     Running   0          5s
```

3. kubectl deployment

```
~/Kishor/UNI/SEM VI/VSEC practicals/8                    INT x  20:41:23
 kubectl get deployments
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
ai-model-deployment  3/3     3            3           90s
```